Weather-Driven Electricity Demand Prediction Pipeline

Maura Sweeney

CPSC 482 Final Project

10 May 2026

1. Project overview and goals

This project constructs a six step end-to-end data pipeline on Google Cloud Platform that ingests six years of hourly weather and electricity data, from Open-Meteo's Historical Weather API and the EIA Hourly Electricity Grid Monitor, respectively. These are joined across geographic regions, trains regression models to predict electricity demand, and showcases results in an interactive Looker Studio dashboard.

The main goal was to determine how reliably weather variables (temperature, humidity, solar radiation, wind speed, and derived heating/cooling degree hours) can predict hourly electricity demand in various US balancing-authority regions, and if a model trained on 2018–2021 data can accurately predict unseen years.

The EIA Grid operator issues a day-ahead demand forecast to schedule generation and avoid under-supply (outages) and over-supply (wasted capacity and cost). According to the EIA, weather is a hugely influential factor in both residential and commercial electricity consumption. Heating, cooling, and lighting loads all respond directly to temperature, solar irradiance, and humidity. A data pipeline that continuously ingests public weather observations and trains on historical demand records could add to or validate utility-issued forecasts, especially for the smaller balancing authorities that report higher forecast error rates.

My goals in this project were to ingest millions of city-hour weather records and millions of BA-region-hour electricity records from January 2018 through December 2023. Those sources were then joined geographically using a city-to-BA lookup table, and multiple city observations are aggregated into population-weighted regional weather features. Next, I trained a linear regression baseline and a boosted tree regressor in BigQuery ML, which were evaluated on 2022 data and tested on 2023 data. Finally, I built a Looker Studio dashboard showing actual vs. predicted demand, model performance by region, feature importance rankings, and more demonstrations of the pipeline's prediction ability.

2. Dataset description

I joined two datasets in this project– Open-Meteo Historical Weather API and EIA Hourly Electricity Grid Monitor. Open-Meteo aggregates reanalysis output from ERA5 (ECMWF) and national weather service models to produce hourly, grid-point weather estimates around the world. It is free for educational and recreational use and requires no API key. The data that I ingested covered 50 US cities from January 2018 to December 2023. I chose cities that would coincide well with the balancing authorities represented in the EIA dataset. Population weighting is applied when aggregating multiple cities in the same BA region, so that higher demand centers contribute more heavily than rural ones. The variables I pulled were temperature_2m, apparent_temperature, relative_humidity_2m, wind_speed_10m, precipitation,

shortwave_radiation, cloud_cover, weather_code, and there was a very low amount of null values, mainly coastal and high-elevation grid cells during extreme weather events.

For the second dataset, the U.S. Energy Information Administration (EIA) publishes near real-time hourly electricity demand, day-ahead forecast demand, and net generation for all reporting balancing authorities via the Open Data API v2. This covers 25 balancing authorities over the same dates January 2018 to December 2023. The fields I used were demand (MWh), day-ahead demand forecast (MWh), and net generation (MWh). There was a slightly higher null rate in this dataset because in smaller BAs there are some reporting gaps, which the EIA handles via seasonal median imputation.

Both sets of data were initially in JSON format, which I converted to CSV before writing to GCS.

The weather data is measured at city latitude and longitude points and electricity demand data is reported at balancing-authority region level. These did not correspond one-to-one since one BA may cover a multi-state area, while a city-point weather observation covers a GPS coordinate.

I used a lookup table to map the 50 cities to a BA region with a population weight. The PySpark join joins all weather observations in a BA into one population-weighted hourly record before joining to the EIA data. The joined fact table contains millions of rows with 40 columns (the weather features, demand/generation values, engineered time features, split assignments, and geographic keys)

3. Project Implementation

The pipeline I implemented flows from raw data into Cloud Storage, which is processed by Spark on Dataproc, and is served from BigQuery for both analytics and ML training. Data is ingested via the Cloud Shell from Open-Meteo API and EIA Hourly API. This data is then stored in a Google Cloud bucket. To process this data in Dataproc, I used a python file which joined datasets using a city to BA lookup, handled null values with seasonal medians, capped values to reasonable levels so invalid data wasn't included, and performed feature engineering, all of which is then stored in a new bucket. Finally, I produced fact and dimension tables using SQL queries in BigQuery, as well as BigQuery ML regression models (linear regression and boosted tree models). I visualized all of these results and interesting values in Looker Studio. This pipeline kept ingestion and processing decoupled and stored cheaply, additionally, all of these functions were able to be used natively in Google Cloud making the process very straightforward.

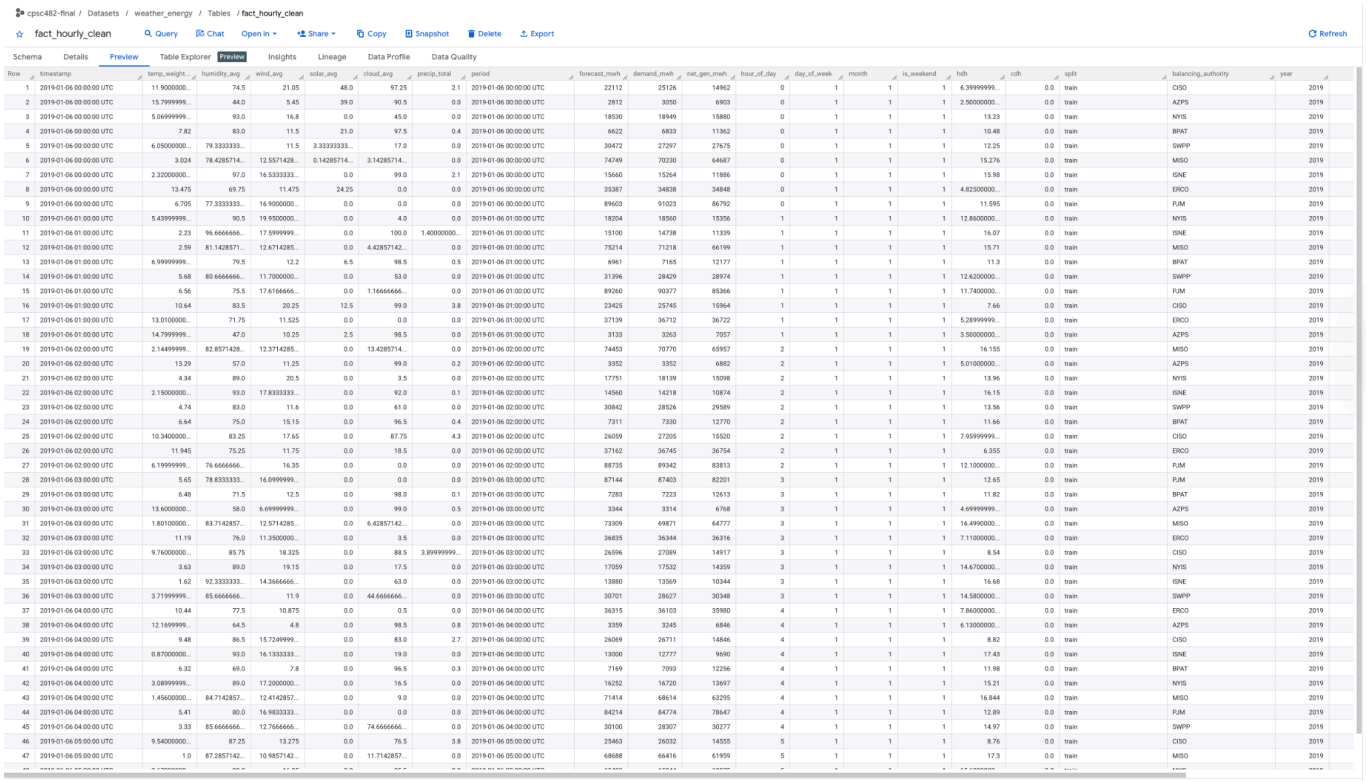
The two scripts which hand ingestion are input_weather.py and inputer_eia.py. Both write incrementally to GCS, one file per city or BA, to avoid holding the full dataset in memory and to allow partial re-runs after rate-limit failures. The EIA API caps responses at 5,000 records per call to prevent going over the API call limit. The script loops until a response signaling completion is received, then writes the concatenated rows to GCS. When they are initially input,

weather data is stored in GCS buckets with one file per city and EIA data is stored with one file per BA. After processing data is stored in Parquet files partitioned by BA and year.

The data is processed via the transform.py job, which runs on a 2-worker Dataproc cluster. First, as sources are read, timestamps are cast using to_timestamp() to align the UTC format used by Open-Meteo with the period strings used by EIA. Column names are normalised.

A DataFrame encodes the mapping from city to BA region together with population weight. Weather observations are then joined to this table, then grouped by balancing authority and timestamp with weighted-average aggregations.

EIA records contain 2–5% nulls in smaller BAs due to reporting gaps. These are imputed using a seasonal hourly median (a Spark window is partitioned by BA, calendar month, and hour-of-day, and null values are replaced with the 50th percentile in that window). This preserves seasonal and demand patterns.

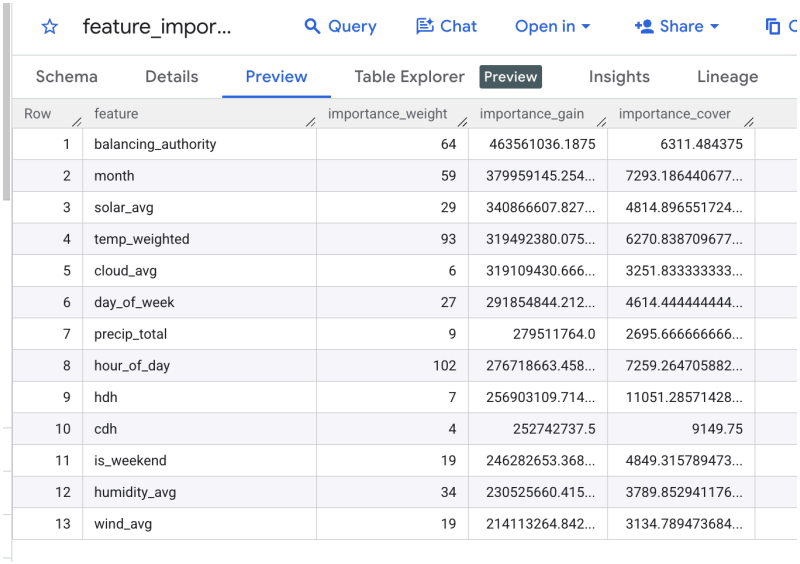


The processed Parquet files are loaded into BigQuery with timestamp-based partitioning and clustering based on balancing authority.

5. Results Obtained

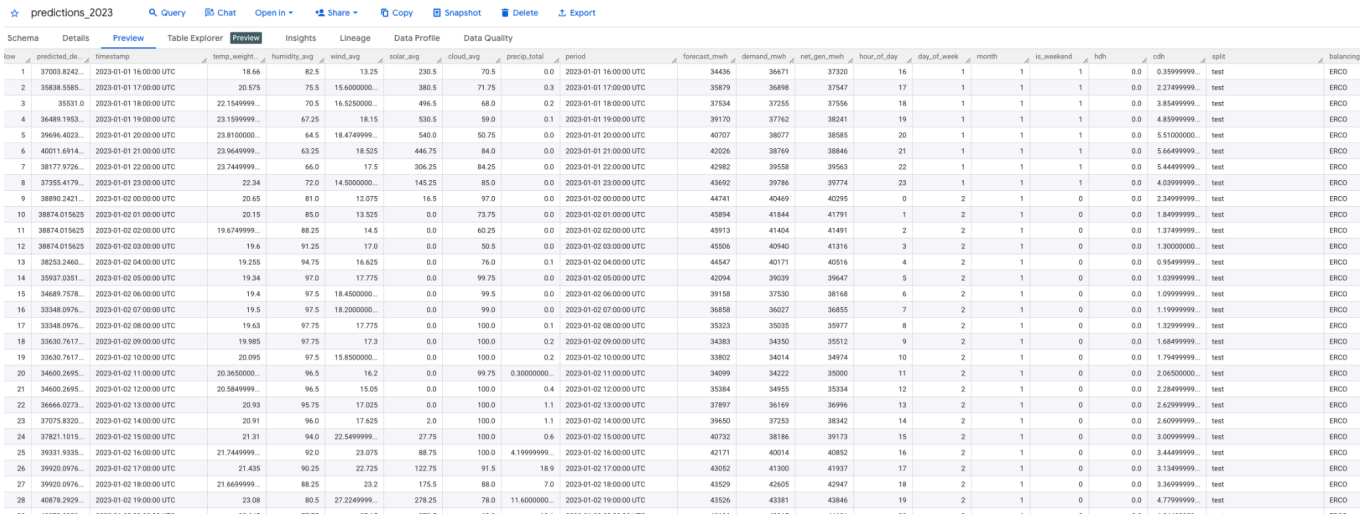
Both the linear regression and boosted tree models were evaluated on the 2022 holdout set using BigQuery ML's ML.EVALUATE function. The boosted tree regressor outperformed the linear regression baseline across all metrics, achieving an R^2 of 0.9899, a mean absolute error of 1,919 MWh, and a mean squared error of 9,296,965, compared to the linear regression's R^2 of

0.9459, MAE of 4,557 MWh, and MSE of 49,755,419. The boosted tree's superior performance indicates that the relationship between weather variables and electricity demand contains meaningful non-linear relationships that a linear model cannot capture. The threshold style of heating and cooling loads where energy consumption rises significantly once temperature crosses certain thresholds works well with tree-based splits. Both models still achieved high R^2 values, confirming that weather features alone explain the large majority of variance in hourly electricity demand across all nine balancing authority regions.



Row	feature	importance_weight	importance_gain	importance_cover
1	balancing_authority	64	463561036.1875	6311.484375
2	month	59	379959145.254...	7293.186440677...
3	solar_avg	29	340866607.827...	4814.896551724...
4	temp_weighted	93	319492380.075...	6270.838709677...
5	cloud_avg	6	319109430.666...	3251.833333333...
6	day_of_week	27	291854844.212...	4614.444444444...
7	precip_total	9	279511764.0	2695.666666666...
8	hour_of_day	102	276718663.458...	7259.264705882...
9	hdh	7	256903109.714...	11051.28571428...
10	cdh	4	252742737.5	9149.75
11	is_weekend	19	246282653.368...	4849.315789473...
12	humidity_avg	34	230525660.415...	3789.852941176...
13	wind_avg	19	214113264.842...	3134.789473684...

Feature importance analysis on the boosted tree model, computed via ML.FEATURE_IMPORTANCE and ranked by importance gain, showed that balancing_authority was the single most influential feature, followed by month, solar_avg, temp_weighted, and cloud_avg. This reflects the large differences in demand scale across regions: PJM averaging roughly 150,000 MWh per hour versus BPAT averaging around 6,500 MWh. The model must account for this before any weather signal becomes meaningful. The month ranked second shows the seasonal baseline load pattern that is independent from day-to-day weather variation. Among weather variables, solar radiation ranked above temperature, suggesting that irradiance is a strong proxy for both daytime warming and the behavioral drivers of cooling load. CDH and HDH ranked lower than raw temperature and solar radiation, indicating that while the degree-hour features add value, the model extracts more meaning from the underlying weather measurements directly.



predictions_2023 Query Chat Open in Share Copy Snapshot Delete Export

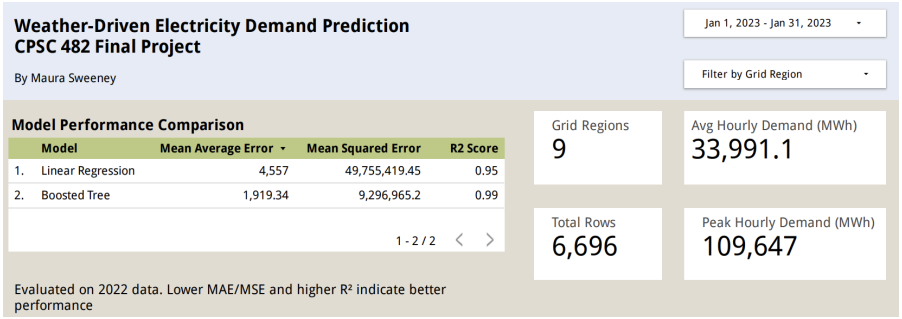
Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data Quality

row	predicted_de...	timestamp	temp_weight...	humidity_avg	wind_avg	solar_avg	cloud_avg	precip_total	period	forecast_mwh	demand_mwh	net_gen_mwh	hour_of_day	day_of_week	month	is_weekend	hdh	cdh	split	balancing
1	37003.8242...	2023-01-01 16:00:00 UTC	18.66	82.5	13.25	230.5	70.5	0.0	2023-01-01 16:00:00 UTC	34436	36671	37320	16	1	1	1	0.0	0.35999999...	test	ERCO
2	35838.5585...	2023-01-01 17:00:00 UTC	20.575	75.5	15.6000000...	380.5	71.75	0.3	2023-01-01 17:00:00 UTC	35879	36898	37547	17	1	1	1	0.0	2.27499999...	test	ERCO
3	35531.0	2023-01-01 18:00:00 UTC	22.1549999...	70.5	16.5250000...	496.5	68.0	0.2	2023-01-01 18:00:00 UTC	37534	37255	37556	18	1	1	1	0.0	3.85499999...	test	ERCO
4	36489.1953...	2023-01-01 19:00:00 UTC	23.1599999...	67.25	18.15	530.5	59.0	0.1	2023-01-01 19:00:00 UTC	39170	37762	38241	19	1	1	1	0.0	4.85999999...	test	ERCO
5	39696.4023...	2023-01-01 20:00:00 UTC	23.8100000...	64.5	18.4749999...	540.0	50.75	0.0	2023-01-01 20:00:00 UTC	40707	38077	38585	20	1	1	1	0.0	5.51000000...	test	ERCO
6	40011.6914...	2023-01-01 21:00:00 UTC	23.9649999...	63.25	18.525	446.75	84.0	0.0	2023-01-01 21:00:00 UTC	42026	38769	38846	21	1	1	1	0.0	5.66499999...	test	ERCO
7	38177.9726...	2023-01-01 22:00:00 UTC	23.7449999...	66.0	17.5	306.25	84.25	0.0	2023-01-01 22:00:00 UTC	42982	39558	39563	22	1	1	1	0.0	5.44499999...	test	ERCO
8	37355.4179...	2023-01-01 23:00:00 UTC	22.34	72.0	14.5000000...	145.25	85.0	0.0	2023-01-01 23:00:00 UTC	43692	39786	39774	23	1	1	1	0.0	4.03999999...	test	ERCO
9	38890.2421...	2023-01-02 00:00:00 UTC	20.65	81.0	12.075	16.5	97.0	0.0	2023-01-02 00:00:00 UTC	44741	40469	40295	0	2	1	0	0.0	2.34999999...	test	ERCO
10	38874.015625	2023-01-02 01:00:00 UTC	20.15	85.0	13.525	0.0	73.75	0.0	2023-01-02 01:00:00 UTC	45894	41844	41791	1	2	1	0	0.0	1.84999999...	test	ERCO
11	38874.015625	2023-01-02 02:00:00 UTC	19.6749999...	88.25	14.5	0.0	60.25	0.0	2023-01-02 02:00:00 UTC	45913	41404	41491	2	2	1	0	0.0	1.37499999...	test	ERCO
12	38874.015625	2023-01-02 03:00:00 UTC	19.6	91.25	17.0	0.0	50.5	0.0	2023-01-02 03:00:00 UTC	45506	40940	41316	3	2	1	0	0.0	1.30000000...	test	ERCO
13	38253.2460...	2023-01-02 04:00:00 UTC	19.255	94.75	16.625	0.0	76.0	0.1	2023-01-02 04:00:00 UTC	44547	40171	40516	4	2	1	0	0.0	0.95499999...	test	ERCO
14	35937.0351...	2023-01-02 05:00:00 UTC	19.34	97.0	17.775	0.0	99.75	0.0	2023-01-02 05:00:00 UTC	42094	39039	39647	5	2	1	0	0.0	1.03999999...	test	ERCO
15	34689.7578...	2023-01-02 06:00:00 UTC	19.4	97.5	18.4500000...	0.0	99.5	0.0	2023-01-02 06:00:00 UTC	39158	37530	38168	6	2	1	0	0.0	1.09999999...	test	ERCO
16	33348.0976...	2023-01-02 07:00:00 UTC	19.5	97.5	18.2000000...	0.0	99.0	0.0	2023-01-02 07:00:00 UTC	36858	36027	36855	7	2	1	0	0.0	1.19999999...	test	ERCO
17	33348.0976...	2023-01-02 08:00:00 UTC	19.63	97.75	17.775	0.0	100.0	0.1	2023-01-02 08:00:00 UTC	35323	35035	35977	8	2	1	0	0.0	1.32999999...	test	ERCO
18	33630.7617...	2023-01-02 09:00:00 UTC	19.985	97.75	17.3	0.0	100.0	0.2	2023-01-02 09:00:00 UTC	34383	34350	35512	9	2	1	0	0.0	1.68499999...	test	ERCO
19	33630.7617...	2023-01-02 10:00:00 UTC	20.095	97.5	15.8500000...	0.0	100.0	0.2	2023-01-02 10:00:00 UTC	33802	34014	34974	10	2	1	0	0.0	1.79499999...	test	ERCO
20	34600.2695...	2023-01-02 11:00:00 UTC	20.3650000...	96.5	16.2	0.0	99.75	0.30000000...	2023-01-02 11:00:00 UTC	34099	34222	35000	11	2	1	0	0.0	2.06500000...	test	ERCO
21	34600.2695...	2023-01-02 12:00:00 UTC	20.5849999...	96.5	15.05	0.0	100.0	0.4	2023-01-02 12:00:00 UTC	35384	34955	35334	12	2	1	0	0.0	2.28499999...	test	ERCO
22	36666.0273...	2023-01-02 13:00:00 UTC	20.93	95.75	17.025	0.0	100.0	1.1	2023-01-02 13:00:00 UTC	37897	36169	36996	13	2	1	0	0.0	2.62999999...	test	ERCO
23	37075.8320...	2023-01-02 14:00:00 UTC	20.91	96.0	17.625	2.0	100.0	1.1	2023-01-02 14:00:00 UTC	39650	37253	38342	14	2	1	0	0.0	2.60999999...	test	ERCO
24	37821.1015...	2023-01-02 15:00:00 UTC	21.31	94.0	22.5499999...	27.75	100.0	0.6	2023-01-02 15:00:00 UTC	40732	38186	39173	15	2	1	0	0.0	3.00999999...	test	ERCO
25	39331.9335...	2023-01-02 16:00:00 UTC	21.7449999...	92.0	23.075	88.75	100.0	4.19999999...	2023-01-02 16:00:00 UTC	42171	40014	40852	16	2	1	0	0.0	3.44499999...	test	ERCO
26	39920.0976...	2023-01-02 17:00:00 UTC	21.435	90.25	22.725	122.75	91.5	18.9	2023-01-02 17:00:00 UTC	43052	41300	41937	17	2	1	0	0.0	3.13499999...	test	ERCO
27	39920.0976...	2023-01-02 18:00:00 UTC	21.6699999...	88.25	23.2	175.5	88.0	7.0	2023-01-02 18:00:00 UTC	43529	42605	42947	18	2	1	0	0.0	3.36999999...	test	ERCO
28	40878.2929...	2023-01-02 19:00:00 UTC	23.08	80.5	27.2249999...	278.25	78.0	11.6000000...	2023-01-02 19:00:00 UTC	43526	43381	43846	19	2	1	0	0.0	4.77999999...	test	ERCO

Applying the trained boosted tree model to the 2023 test set via ML.PREDICT produced the predictions_2023 table with 43,824 hourly records across the nine balancing authorities. A preview of the predictions shows that predicted_demand_mwh tracks actual demand_mwh closely on an hour-by-hour basis. The predictions table also retains the EIA day-ahead forecast (forecast_mwh) for direct comparison, enabling the Looker Studio dashboard to visualize all three together. The close alignment between predicted and actual demand across the test year validates that the pipeline generalizes well to unseen data, with the model trained entirely on 2018–2021 observations successfully capturing the demand patterns of a held-out year.

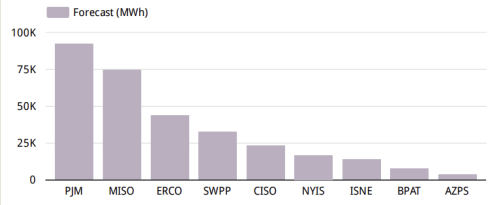
6. Visualizations/dashboard

The Looker Studio Dashboard connects directly to the BigQuery dataset and addresses my main goal in this project through scorecards, charts, and tables.



The pipeline's results are shown across a single page with interactive filter controls for grid region and date range. The dashboard includes four scorecards at the top: grid regions covered (9), total rows (6,696 for the filtered view), average hourly demand (33,991 MWh), and peak hourly demand (109,647 MWh), providing an at-a-glance summary of the dataset's scope and scale. A model performance comparison table displays the evaluation metrics for both models side by side, evaluated on 2022 holdout data, confirming that the boosted tree (R^2=0.99, MAE=1,919 MWh) outperformed the linear regression baseline (R^2=0.95, MAE=4,557 MWh).

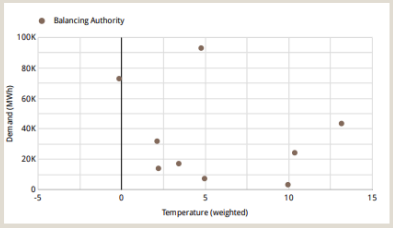
Average Electricity Demand by Grid Region (MWh)



PJM and MISO are the largest grid regions by demand volume, showing how they are more dense population centers.

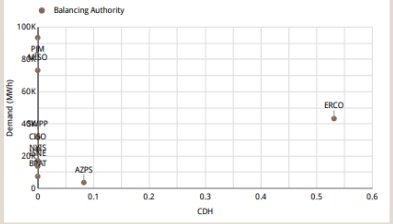
The average electricity demand by grid region bar chart confirms that PJM and MISO are by far the largest consuming regions, reflecting their dense population centers. This scale difference (PJM averaging roughly 90,000 MWh versus BPAT averaging under 10,000 MWh) explains why balancing authority ranked as the top feature in the importance analysis.

Temperature vs. Electricity Demand



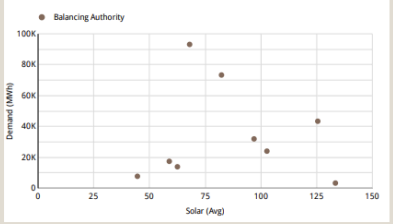
Population weighted regional temperature plotted against hourly demand. Each point represents a balancing authority

Cooling Degree Hours vs. Electricity Demand



CDH shows temperatures that are above threshold, influencing air conditioning load. Higher CDH correlates with higher summer demand.

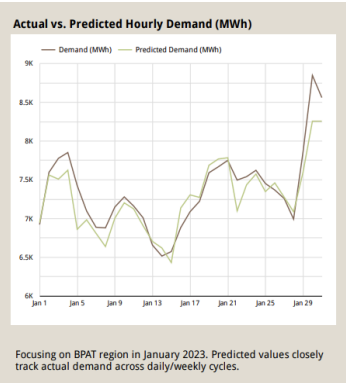
Solar Energy vs. Electricity Demand



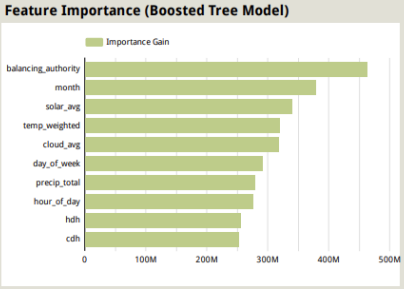
Average shortwave radiation (solar energy) plotted against demand. Higher solar irradiance linked to warmer conditions, higher cooling loads.

The three scatter plots highlight the relationships between weather inputs and electricity demand. The temperature versus demand chart shows a non-linear U-shaped pattern consistent with heating and cooling loads on either side of a "comfort threshold". The cooling degree hours versus demand chart shows how demand changes when the temperature goes above a certain threshold, with ERCO (Texas) appearing as an outlier at the far right due to its hot climate. The solar radiation versus demand chart reveals a positive relationship, with higher irradiance corresponding to elevated demand due to its relationship with warm daytime temperatures.

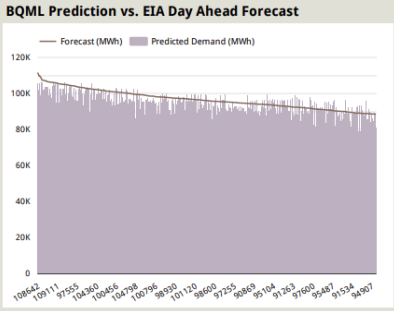
The actual versus predicted hourly demand time series, filtered to the BPAT region for January 2023, shows that the boosted tree model closely tracks the real demand signal across daily and weekly cycles. The predicted values follows a pattern of lower overnight demand, rising through the morning, peaking in the evening, and showing the week-over-week variation across the month. The feature importance chart points to the ranking identified in the ML.FEATURE_IMPORTANCE query, with balancing authority and month dominating, followed by solar radiation, temperature, and cloud cover. Finally, the BQML prediction versus EIA day-ahead forecast chart compares the model's predictions against the utility-issued forecasts for the 2023 test year. Both series closely match across the demand range of roughly 80,000 to 120,000 MWh for the larger regions. This proves that the pipeline's model produces forecasts that are of comparable quality to professional utility forecasts.



Focusing on BPAT region in January 2023. Predicted values closely track actual demand across daily/weekly cycles.



Balancing authority and month are the strongest predictors, showing differences in demand per region and seasonal patterns.



Comparison of model predictions against EIA's official day-ahead forecast across 2023 test data.

7. Challenges Encountered and Resolutions

Initially, I encountered an issue with Open-Meteo rate limiting after many sequential requests. My first bulk run failed after about 20 cities with that error code, so I added time.sleep(0.5) between city requests and implemented an easy retry method by checking if GCS file already exists before querying. The EIA Hourly Electricity Grid Monitor API caps responses at 5,000 records per call, but a single balancing authority spans up to 52,560 hours across three data series (demand, day-ahead forecast, and net generation), generating over 157,000 rows per BA. To handle this, the ingestion script implements a while-loop that increments an offset parameter by 5,000 on each iteration and terminates only when the API returns an empty response, ensuring all records are collected via as many calls as needed. Another issue came from the geographic mismatch between the two data sources: weather observations are recorded at city-level latitude and longitude coordinates, while electricity demand is reported at the balancing authority region level with no clear mapping between the two. To resolve this, I created a lookup table by cross-referencing NERC region maps and EIA balancing authority boundary data, assigning each of the 50 weather cities to a corresponding BA region and a population-based weight. This table was encoded as a PySpark DataFrame and used as the basis for the geographic join in the transformation script, enabling multiple city-level weather observations to be joined into a single population-weighted feature before being joined to the EIA demand records.

8. Lessons learned

I learned that writing one file per source (one CSV per city, one per BA) instead of one big file made it much easier to ingest data. When individual API calls failed partial re-runs only needed to re-fetch the missing files rather than restart the entire process.

The geographic alignment problem turned out to be the most time-intensive part of the pipeline, not any GCP service. The technical Spark and BigQuery work was straightforward once the lookup table was correct, and an incorrect city-to-BA mapping would produce wrong regional weather features that were not obviously incorrect.

The boosted tree significantly outperformed the linear regression baseline, confirming that the relationship between weather and demand is non-linear since cooling and heating loads have threshold effects that linear models underfit. However, the 2023 ERCOT outlier highlights a limitation: a model trained on historical data cannot anticipate demand spikes driven by climate events outside the training distribution.

Finally, creating a denormalised view in BigQuery before connecting Looker Studio simplified the dashboard creation significantly. Looker Studio's blended data sources are useful but not as intuitive and quick as an SQL query. A single flat view with all needed columns is the easiest starting point.

9. Potential Next Steps

While trying to figure out how to join the datasets on the region, I found some built in BigQuery geography functions. It might be beneficial to replace the manual city-to-BA lookup table with a join using BigQuery Geography functions and the official EIA BA boundary GeoJSON, which would allow any latitude and longitude point to be assigned to its correct region automatically.

Also, it would be interesting to create models for each unique BA rather than a single global model; regional BAs have unique demand curves that a general model may underfit.

I would also like to set up BigQuery scheduled queries and Scheduler to compute daily summary statistics and refresh a monitoring table that tracks possible errors in the model and continuously ingests data and updates predictions regularly.

Finally, extending the model to cover to all 70 EIA-reporting BAs and the full set of corresponding weather cities, scaling the fact table significantly to cover the rest of the world outside of the United States, would be interesting.